

COMPUTER SCIENCE TRIPOS Part IB – 2020 – Paper 4

3 Compiler Construction (tgg22)

Function closure (a) In the context of the compilation of functions, what is a *closure*? [2 marks]

Answer: A closure is a data structure used to represent a function together with the values of the free variables of the function's body. This can be used for languages that allow functions to be defined at any nesting depth. Closures can be implemented in the heap, containing the address of the beginning of the function's code as well as values of the free variables (an "environment"). These values may be pointers to other objects in the heap.

Jargon VM (b) The front-end of our Slang compiler eliminates let-bindings by replacing the code

```
let x e1 in e2 end
```

with the code

```
(fun x -> e2 end) e1
```

Apply this transformation to the following Slang code.

```
let f(x) =  
  let x1 = e1  
  in let x2 = e2  
    in e3 end  
  end  
in  
  e  
end
```

[3 marks]

Answer: The result is

```
(fun f -> e)  
  (fun x -> (fun x1 -> (fun x2 -> e3) e2) e1)
```

Function closure (c) Describe the structure of the Jargon code generated from the Slang in your answer to Part (b). (Don't worry about getting the syntax exactly right.) [6 marks]

Answer: To make things clearer, rewrite the answer to question (b), making the possible free variables explicit in the expressions:

```
(fun f -> e(f))  
  (fun x -> (fun x1 -> (fun x2 -> e3(x2, x, x1)) e2(x1, x)) e1(x))
```

Note that the first variable in each list is the associated function's parameter, found on the stack, while the others are found in the function's closure in the heap.

Recall that the compilation of `(fun x -> e') e''` will result in code that

- (i) evaluates `e''` and pushes the resulting value on the stack
- (ii) creates a closure for `(fun x -> e')` in the heap and pushes a pointer to it on the stack. The function that creates a closure needs (at least) two arguments (c, k) , where c is the address of the first instruction in `e'` and the number of free variables of `e'` (not including x). This will cause k values to be popped off the stack and placed in the closure.
- (iii) applies the top of the stack to the argument below.

So the code generated will have the following form for some code locations c, c_0, c_1, c_2, c_3 .

```

c :   create closure (c1, 0) for (fun x -> ...)
      create closure for (c0, 0) for (fun f -> e)
      apply
      return
      halt
c0:   code for e(f)
      return
c1:   code for e1(x)
      fetch value of x from stack to put in closure
      create closure (c2, 1) for (fun x1 -> ...)
      apply
      return
c2:   code for e2(x1, x)
      fetch value of x1 to from stack put in closure
      fetch value of x to from current closure on the heap to put in closure
      create closure (c3, 2) for (fun x2 -> e3)
      apply
      return
c3:   code for e3(x2, x, x1)
      return

```

- Function closure (d) Consider the Jargon code generated in Part (c). Suppose the function `f` is called with the value v somewhere in the code generated from the expression `e`. Describe what happens at runtime when `f(v)` is executed. In particular, describe the closures that exist in the heap and how they are used to evaluate `f(v)`. [4 marks]

Answer: In the body of `f` (starting at c_1), the code first leaves the value of `e1(v)` on the stack. (The value v for `x` is found by an index off the frame pointer.) Call this value v_1 . Then value v is fetched and a closure is created containing (at least) (c_2, v) . This closure is then called with argument v_1 . The code at c_2 evaluates `e2(x1, x)` to some value v_2 , leaving it on the stack. Values associated with variables `x1`, and `x` are fetched and pushed on the stack in order to create the closure $(c_3, 2)$. Then this closure is called with the argument v_2 . Thus we have at least three closures being created on the heap.

- Function closure (e) Describe a better way of compiling let-bindings such as those associated with `x1` and `x2` in the code above. Rather than creating closures, the idea is to include these “local variables” in the stack frame for `f`. Explain in detail how this might be done. [5 marks]

Answer:

If a function’s argument is “below” the function’s stack frame, then place “local variables”

directly above the entries in the stack frame for return, saved frame pointer, and return address.

```
c :  create closure for (c1, 0) for f
      create closure for (c0, 0) for (fun f -> e)
      apply
      halt
c0:  code for e(f)
      return
c1:  code for e1(x) (leave value on top of stack)
      code for e2(x1, x) (leave value on top of stack)
      code for e3(x2, x, x1) (find all variables as index off of fp)
      return
```

Clearly this is an improvement in that it uses the stack instead of the heap to store the temporary values associated with “local variables”.
